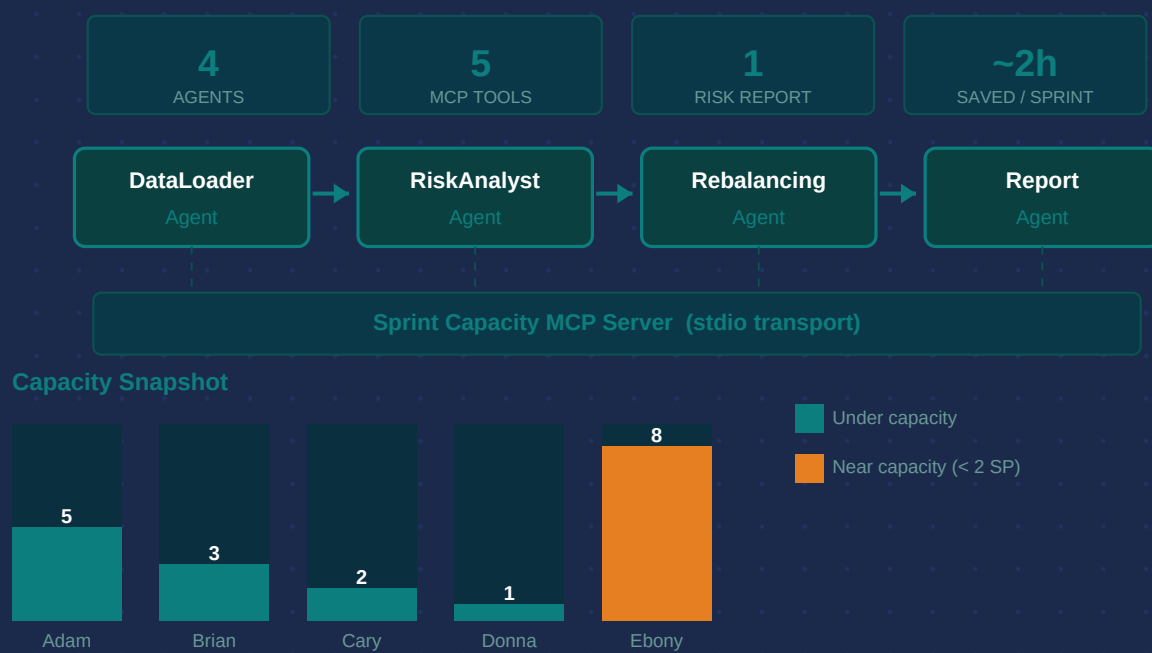


Sprint Capacity Planning

Multi-Agent AI System

Autonomous risk detection and rebalancing for agile sprint teams

Google ADK · Model Context Protocol · Gemini · Python · pandas



SUBMITTED BY

Author Name

LinkedIn Profile URL

1. The Business Problem

Software delivery is one of the largest controllable cost centres in any technology enterprise. Engineering salaries, contractor fees, and cloud infrastructure collectively represent **40–70% of a typical tech company's operating expenditure**. Yet the question that drives how that cost is deployed — *how much can each person actually deliver this sprint?* — is still answered manually, in a spreadsheet, every two weeks.

This is not a minor inconvenience. Mis-calibrated sprint capacity has a direct financial cost in two directions:

- **Over-commitment inflates cost without delivering value.** When a team is assigned more story points than it can complete, work rolls over into the next sprint. Rolled-over work means unplanned overtime, re-planning sessions, delayed product releases, and — in customer-facing products — lost revenue from features that miss a launch window.
- **Under-commitment wastes paid capacity.** When available hours are not filled with meaningful work, the enterprise pays for engineering time that produces no output. Even a 10% utilisation gap across a 50-person team represents hundreds of thousands of dollars in idle salary per year.
- **Manual planning is itself a cost.** Sprint planning ceremonies in large organisations can consume 2–4 hours per team per fortnight. With multiple teams, that is days of senior engineering and management time spent on arithmetic that a machine can do in seconds.

The root cause is structural: the data needed to make the right decision — leave calendars, holiday schedules, velocity history, and committed story points — lives in different tools and is assembled by hand each cycle. No enterprise should be making cost-critical resource decisions this way in 2025.

2. The Central Idea and Innovation

The central idea is simple: **replace the spreadsheet with an autonomous AI agent that reasons over workforce data, calculates real capacity, and surfaces actionable risk — without human intervention.**

The innovation is not in the capacity formula itself (subtraction is not novel). The innovation is in the **multi-agent architecture**: using Google ADK to build a pipeline where four specialised agents coordinate through shared session state, each calling tools exposed by a local MCP server — doing exactly what they are best at, nothing more.

This separation is the key design insight. Language models are poor at precise arithmetic but excellent at reading structured data and producing contextually aware, human-readable risk narratives. Python is the opposite. By combining them inside a multi-agent ADK pipeline — with tools exposed through MCP — we get output that is both **numerically exact** and **immediately actionable**.

The agent is also meaningfully autonomous: given a path to a data file, it independently decides what to read, what to compute, how to format the result, and what risk language to use. It does not wait for prompts at each step. A scrum master triggers it once and receives a complete decision-support document. That is the enterprise value proposition.

3. The Solution

We built a **four-agent system using Google ADK** — a *SequentialAgent* orchestrating four specialised AI agents, each consuming tools from a local **MCP server**. The system ingests sprint workforce data from Excel, computes per-employee capacity, identifies delivery risks, suggests rebalancing, and produces a stand-up-ready risk report — all from a single command.

The agent pipeline operates as follows:

- **DataLoaderAgent** calls the MCP server's `load_sprint_data` tool to read the Excel workbook and compute capacity metrics. Confirms sprint configuration and writes results to a shared cache.
- **RiskAnalystAgent** calls three MCP tools in sequence: team overview, risk identification, and individual employee details for every flagged person. It reasons over the data to produce a structured risk summary.
- **RebalancingAgent** reads the risk findings and — for any over-capacity employee — calls the MCP server to identify the best recipient for redistributed story points.
- **ReportAgent** synthesises all findings from shared session state into a concise, actionable sprint risk report. No MCP tools needed — it reasons entirely from what the previous agents wrote.

The output replaces a two-hour planning ceremony with a seconds-long agent run. The scrum master's role shifts from data gatherer to decision maker — which is where their value lies.

4. Why Google ADK and MCP?

This problem could be solved with a Python script that calls five functions in sequence. We chose Google ADK and MCP deliberately, and the reasons matter for enterprise deployment.

4.1 Multi-agent separation of concerns

Sprint capacity planning is not a single calculation — it is a **multi-step reasoning process**: load data, identify risks, suggest rebalancing, synthesise a report. Each step has different requirements and different tools. A *SequentialAgent* maps directly onto this structure: each sub-agent has exactly one responsibility, one tool set, and one output key. A flat script conflates all concerns, making each step harder to audit, test, or swap independently.

4.2 Session state eliminates coordination overhead

Google ADK gives every agent access to shared session state. Each agent writes its output to a named `output_key` and the next agent reads it via a `{state_key}` placeholder in its instruction. There are no function signatures to maintain, no argument threading to break, and no global variables to race. This is critical for enterprise pipelines maintained by rotating teams.

4.3 SequentialAgent — deterministic orchestration without LLM cost

Because the pipeline steps always run in the same order and each clearly depends on the previous output, we use **SequentialAgent** rather than an LLM orchestrator. Deterministic execution means no hallucinated routing decisions, no extra model calls, and predictable latency. An LLM is reserved for the reasoning steps where language and judgement actually matter.

4.4 MCP server — decoupled, reusable tool layer

All five capacity tools live in a standalone **MCP server** (`mcp_server.py`) that communicates via the Model Context Protocol. This decoupling means: any agent framework that speaks MCP can consume these tools without code changes; each agent gets only the tools it needs via *tool_filter* (principle of least privilege); and the tool implementation can be tested, updated, or replaced without touching the agent code.

4.5 Modularity enables enterprise extension

Because each agent is self-contained and all tools are in the MCP server, replacing the data source from Excel to Jira means adding one MCP tool. Adding a Slack output means adding a fifth ADK agent. The core pipeline — SequentialAgent orchestration, shared state, MCP tools — is untouched. This is the extensibility model enterprises need: upgradeable without a full rewrite.

5. Architecture

The system uses a **SequentialAgent** (Google ADK) to orchestrate four specialised AI agents. Each agent calls tools via a local **MCP server** over stdio. Agents communicate through shared session state — each writes its output to a named key and the next reads it via placeholder injection in its instruction.

```
DataLoaderAgent -> RiskAnalystAgent -> RebalancingAgent -> ReportAgent
```

The orchestrator is deterministic — not LLM-driven. Execution order is fixed because data must be loaded before risks can be found, risks before rebalancing, and rebalancing before the final report. Each individual agent is LLM-driven within its own scope, calling MCP tools and reasoning over the results.

Agent	Writes to State	Purpose
DataLoaderAgent	data_status	Calls load_sprint_data via MCP — reads Excel, computes capacity metrics, writes to cache file
RiskAnalystAgent	risk_analysis	Calls get_team_overview, identify_capacity_risks, get_employee_details via MCP — drills into risks
RebalancingAgent	rebalancing_plan	Calls suggest_work_rebalancing via MCP for each over-capacity employee — finds best recipient
ReportAgent	final_report	Synthesises {risk_analysis} and {rebalancing_plan} from shared state — no MCP tools needed

Table 2 — ADK agent pipeline (SequentialAgent orchestration, shared session state)

MCP Tool (Sprint Capacity Server)	Called by	Returns
load_sprint_data(excel_path)	DataLoaderAgent	Employees loaded, sprint config; writes sprint_cache.json
get_team_overview()	RiskAnalystAgent	Team size, total SP, utilisation %, employees at 85%+ capacity
identify_capacity_risks()	RiskAnalystAgent	Employees over capacity or with less than 2 SP of headroom
get_employee_details(name)	RiskAnalystAgent	Full capacity breakdown: available days, max SP, assigned SP, headroom
suggest_work_rebalancing(from, points)	RebalancingAgent	Best recipient ranked by headroom, capacity after transfer, alternatives

Table 3 — MCP tools (`mcp_server.py`, stdio transport; *tool_filter* gives each agent least-privilege access)

5.1 Session state — the shared data contract

Google ADK agents communicate via session state. Each agent writes to an *output_key*; the next agent reads it via a *{state_key}* placeholder in its instruction. This means agents are loosely coupled: each only knows about its own input and output, not about the full pipeline. The session is also pre-seeded with the Excel path so any agent that needs it can reference *{excel_path}* without it being hard-coded.

5.2 MCP tool_filter — least-privilege tool access

Each agent's *McpToolset* is configured with a *tool_filter* that restricts which MCP tools the agent can see. *DataLoaderAgent* only sees *load_sprint_data*; *RebalancingAgent* only sees *suggest_work_rebalancing*. This prevents any agent from calling tools outside its scope — reducing hallucination risk and enforcing the principle of least privilege.

5.3 Capacity formula

The deterministic calculation inside the MCP server (Python, not LLM):

```
available_days = sprint_days - planned_leave - unplanned_leave - holidays
max_sp = available_days x 1.5 (configurable velocity: SP per day)
remaining_sp = max_sp - sp_assigned
```

Velocity (1.5 SP/day) is stored in the Excel Config sheet, not in code, so a scrum master can tune it per team without touching the agent. Status thresholds — Over, At, Under capacity — are evaluated here in Python where arithmetic is exact, not in the language model where it is not.

6. Execution Scenarios and Business Outputs

6.1 Sprint planning (baseline scenario)

At the start of a sprint, the scrum master runs the agent with all unplanned leave set to zero. The agent produces the following capacity table and an AI risk summary in one pass:

Employee	Sprint Days	Planned Leave	Holidays	Avail Days	Capacity %	Max SP	SP Assigned	Remaining SP	Status
Adam	10	2	1	7	70%	10.5	5	5.5	✓ Under
Brian	10	2	1	7	70%	10.5	3	7.5	✓ Under
Cary	10	1	1	8	80%	12.0	2	10.0	✓ Under
Donna	10	1	1	8	80%	12.0	1	11.0	✓ Under
Ebony	10	3	1	6	60%	9.0	8	1.0	■ Watch

Table 1 — Sprint capacity output generated by the agent (1 holiday assumed)

Business reading: Ebony is assigned 8 SP against a maximum of 9 SP — only 1 SP of buffer before she becomes a delivery risk. The agent flags this immediately, allowing the team to redistribute work before the sprint starts rather than discovering the problem at the mid-sprint review.

6.2 Mid-sprint replan: unplanned absence

Cary is unexpectedly absent for 2 days. The scrum master updates *Unplanned_Leaves* to 2 and re-runs the agent in seconds. Cary's available days drop from 8 to 6, but her 2 SP commitment remains well within her revised maximum of 9.0 SP. The AI summary confirms no rebalancing is required. **Without**

the agent, this same assessment would take 20 minutes of manual recalculation.

6.3 Over-commitment detection

If Ebony's committed story points are raised to 10 — exceeding her available maximum of 9.0 — the agent immediately flags **Over Capacity** and the AI risk summary names Donna (11 SP of remaining headroom) as the recommended transfer target. The enterprise value: a delivery risk is caught and resolved at planning time, not at sprint review when the damage is already done.

6.4 Sample agent output after multi-agent investigation

After the four-agent pipeline completes — data loaded, risks identified, rebalancing proposed, report synthesised — the final output is a decision-ready sprint risk report. The key difference from a simple summary: each finding was discovered through real MCP tool calls, reasoned over by a specialised agent, and written into shared state before the ReportAgent assembled the final narrative.

Sprint Risk Assessment: LOW

- **Ebony** has only 1.0 SP of remaining capacity — the tightest margin on the team. Do not assign additional work without rebalancing.
- **Donna** carries the most available headroom (11.0 SP) and is the first-choice recipient for any spill-over or late-added work items.
- **Adam and Brian** are at 70% utilisation with 5.5 and 7.5 SP remaining respectively — comfortable buffers for a two-week sprint.
- **Overall risk: Low.** The team is within safe delivery bounds. No re-planning is required before kickoff. Monitor Ebony's workload at the mid-sprint check-in.

Figure 1 — Agent-generated sprint risk summary (output may vary)

7. Project Journey

7.1 Where it started

The starting point was a real frustration: every sprint, a scrum master was spending 30–45 minutes building the same capacity spreadsheet from scratch — pulling leave data from HR, checking the holiday calendar, and tallying story points from the project board. The first prototype was a flat Python script. It worked, but it was impossible to extend, difficult to test, and broke every time a new requirement appeared.

7.2 Why the multi-agent model was the right answer

Adding an AI summary step to the flat script created immediate problems: the data processing and the language model call were tangled together, making the code hard to read and impossible to test. Migrating to a multi-agent system — with Google ADK orchestrating four specialised agents and an MCP server hosting all tools — gave each concern its own clearly bounded agent. The code became more structured, dramatically easier to understand, and independently testable at each stage.

7.3 Key decisions and lessons

- **Python owns the numbers; the AI owns the narrative.** All arithmetic happens in the MCP server in Python. The LLM only reasons over structured results. This gives exact, auditable calculations and fluent human-readable output — neither of which the other tool reliably provides.
- **MCP as the tool boundary.** Placing all tools in an MCP server rather than directly in the agent code means the tool layer is framework-agnostic. Any future migration to a different agent framework leaves the tools intact.
- **The data file is the interface.** Making the Excel workbook the only user-facing input means non-technical stakeholders — HR, finance, project managers — can contribute data without touching any agent code. This is essential for enterprise adoption.
- **Velocity belongs in config, not code.** Storing the SP/day velocity assumption in the workbook means the MCP server self-adjusts to each team's historical performance without a code change.

7.4 Enterprise roadmap

- **Jira / Azure DevOps integration:** add a *load_jira_sprint* MCP tool so DataLoaderAgent reads live sprint data instead of Excel.
- **Cost modelling:** add daily contractor rates to the workbook and a *compute_financial_risk* MCP tool to quantify the cost of over-commitment.
- **Slack / Teams delivery:** add a fifth ADK agent — SlackReporterAgent — that posts the *final_report* state key to the team's stand-up channel.
- **Persistent sessions:** swap *InMemorySessionService* for *VertexAiSessionService* to persist sprint history across runs and enable velocity trend analysis.

8. Track Relevance and Value Delivered

This submission is built for the **Enterprise AI Agents track**. The problem it solves — workforce capacity mis-allocation — is directly on the cost and revenue axis the track targets. Mis-calibrated sprint capacity delays product releases, wastes engineering salary, and erodes team morale. The agent addresses all three.

The system satisfies the track's core requirements:

- **Compelling business problem:** sprint capacity planning is a fortnightly cost-critical decision made by every software delivery organisation on the planet.
- **AI is central, not decorative:** four AI agents each perform specialised reasoning — risk identification, rebalancing recommendation, and narrative synthesis. Agents decide what to investigate, not a fixed script. That is agentic behaviour on a real business problem.
- **Clear enterprise value:** the system replaces 30–45 minutes of manual work per team per sprint, surfaces over-commitment before it becomes a delivery failure, and provides a reusable, extensible foundation for broader workforce intelligence tooling.
- **Innovation in architecture:** Google ADK multi-agent orchestration combined with a standalone MCP server creates a production-grade separation of concerns — deterministic Python for arithmetic, LLM agents for judgement, MCP for tool interoperability. Each layer is independently replaceable without touching the others.

Built with Google ADK · MCP · Python · pandas · AI-Powered Insights | Kaggle Enterprise AI Agents Track